ASSESSMENT 1: ANALYTICAL PROBLEM SOLVING SOLUTIONS

Course Code: CSA17 Course Name: Artificial Intelligence

Name: YUVANSHANKAR (CO-1, AT-1)

1. Water Jug Problem

Problem: We have a 4-gallon jug and a 3-gallon jug with no measuring marks. We need to get exactly 2 gallons of water in the 4-gallon jug.

State space: (x, y) where x is the water in the 4-gallon jug (0 to 4) and y is the water in the 3-gallon jug (0 to 3).

Initial state: (0, 0)

Goal state: (2, y), y can be anything (mostly (2, 0)).

Steps:

1. Fill the 3-gallon jug: (0, 0) → (0, 3)
2. Pour the water from the 3-gallon jug into the 4-gallon jug: (0, 3) → (3, 0)
3. Fill the 3-gallon jug again: (3, 0) → (3, 3)
4. Pour from the 3-gallon jug into the 4-gallon jug till it becomes full: (3, 3) → (4, 2). The 4-gallon jug needs only 1 more gallon, so the 3-gallon jug is left with 2.
5. Empty the 4-gallon jug on the ground: (4, 2) → (0, 2)
6. Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug: (0, 2) → (2, 0)

Final result: Exactly 2 gallons are obtained in the 4-gallon jug.

2. Mars Rover Analysis

Percepts: Soil composition data from the spectrometer, high-resolution terrain images from the navigation cameras, wheel slip/resistance feedback, solar panel energy level, and atmospheric pressure/temperature.

Environment: Fully observable for the nearby surroundings but only partially observable overall; deterministic for the physical motor actions but stochastic because of unpredictable terrain obstacles; sequential; dynamic; single-agent.

Actions: Drive forward/backward, turn left/right, adjust the camera angle, use the robotic arm, drill into rock samples, store the samples in chambers, and send data back to Earth.

Performance measure: Quantity and quality of the scientific samples collected, energy efficiency, safe navigation without getting stuck, and accurate transmission of telemetry.

Agent architecture: Model-based reflex agent with state. It needs an internal state to track the visited locations, battery status and sample storage capacity, and it handles partial observability by keeping an internal map of the terrain.

3. 8-Queens Problem Formulation

States: Any arrangement of 0 to 8 queens on the chessboard.

Initial state: No queens on the board.

Actions: Place a queen in any empty square.

Transition model: Returns the board with a queen added to the given square.

Goal test: 8 queens are on the board and no two queens attack each other (no two share the same row, column or diagonal).

Search strategy: Can be solved using Backtracking Search (a Depth-First Search variant with constraint propagation) or the Min-Conflicts heuristic local search.

4. OLA Cab Booking Problem Agent & Pseudocode

Agent type: Goal-based agent (and utility-based for cost/comfort optimization). It compares the different paths, cab types (mini, micro, sedan, shared, prime), fares and estimated time of arrival (ETA) to satisfy the user's destination and comfort requirements.

Pseudocode:

```
function OLA-Booking-Agent(user_input) returns selected_cab_and_route
    goal.destination = user_input.destination
    goal.comfort_preference = user_input.cab_type

    available_cabs = GetNearbyCabs(user_input.current_location)
    filtered_cabs = []

    for each cab in available_cabs:
        if cab.type == goal.comfort_preference or cab.type is compatible:
            cost = CalculateFare(cab, goal.destination)
            eta = EstimateTime(cab, goal.destination)
            utility = EvaluateUtility(cost, eta, cab.comfort_rating)
            filtered_cabs.append((cab, utility))

    best_option = SelectMaxUtility(filtered_cabs)
    BookCab(best_option.cab)
    return best_option
end function
```

5. Uniform Cost Search (UCS) on Delivery Network

Graph and edge costs: S → A (1), S → G (12), A → B (3), A → C (1), B → D (3), C → D (1), C → G (2), D → G (3)

Priority queue is written as (Path, Cost).

Step 1 (Initialize): [(S, 0)]

Step 2 (Expand S): Pop (S, 0) and add neighbors A and G. Queue: [(S→A, 1), (S→G, 12)]

Step 3 (Expand S→A): Pop (S→A, 1) and add B (cost 4) and C (cost 2). Queue: [(S→A→C, 2), (S→A→B, 4), (S→G, 12)]

Step 4 (Expand S→A→C): Pop (S→A→C, 2) and add D (cost 3) and G (cost 4). Queue: [(S→A→C→D, 3), (S→A→B, 4), (S→A→C→G, 4), (S→G, 12)]

Step 5 (Expand S→A→C→D): Pop (S→A→C→D, 3) and add G (cost 6). Queue: [(S→A→B, 4), (S→A→C→G, 4), (S→A→C→D→G, 6), (S→G, 12)]

Step 6 (Next minimum): The goal G is reached through S → A → C → G with a total cost of 4.

Optimal path: S → A → C → G with a minimum total cost of 4.